

LangGraph & CrewAI

How they work · When to use which

State graphs · nodes & edges · super-steps · checkpoints · interrupts · agents, tasks & crews · sequential vs hierarchical · Flows

state graph

checkpointer

human-in-the-loop

crew & process

flows

What we cover today



Two mental models

Draw the control flow vs. staff a team

1



LangGraph internals

State · nodes · edges · super-steps · reducers

2



LangGraph in production

Checkpoints · interrupts · time travel · multi-agent

3



CrewAI internals

Agents · tasks · crews · processes

4



CrewAI Flows

Event-driven control around crews

5



Choosing & combining

Side-by-side and decision guide

6

Two mental models for the same problem



LangGraph — you draw the control flow

- Your app is an explicit graph: a shared state plus functions (nodes) wired by edges
- You decide every branch, loop and stop condition
- The LLM lives inside nodes; the structure around it is deterministic code
- Low-level and unopinionated — maximum control



CrewAI — you staff a team

- Your app is a crew: role-playing agents assigned to tasks
- You describe who does what and what “done” looks like
- The framework drives the collaboration between agents
- High-level and opinionated — fastest path to a working prototype

Same goal, different home for control. Both run multi-step LLM work with tools. In LangGraph, control lives in the graph you write; in CrewAI, it lives in the crew’s process — unless you take it back with Flows.



01

LANGGRAPH

Agents as state machines

Three primitives: state, nodes, edges



State

A typed schema (TypedDict or Pydantic) shared by the whole graph. Each key can have a reducer that says how updates merge — e.g. `add_messages` appends instead of overwriting.



Nodes

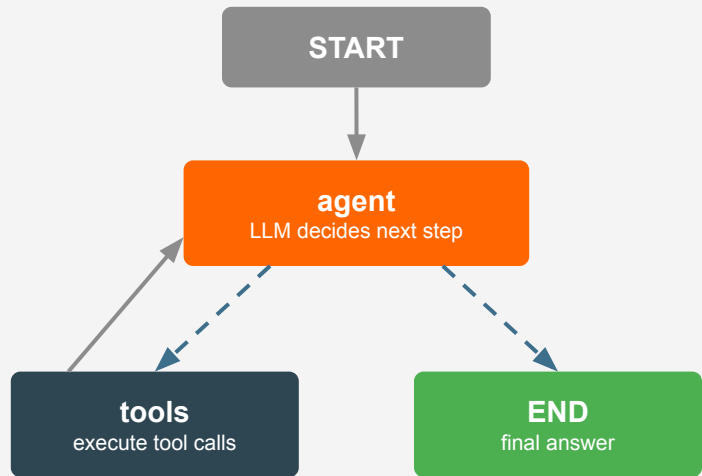
Plain Python functions: take the current state, return a partial update. A node can call an LLM, a tool, an API — or just compute.



Edges

Decide what runs next. Normal edges are fixed; conditional edges call a routing function on the state. START and END mark entry and exit.

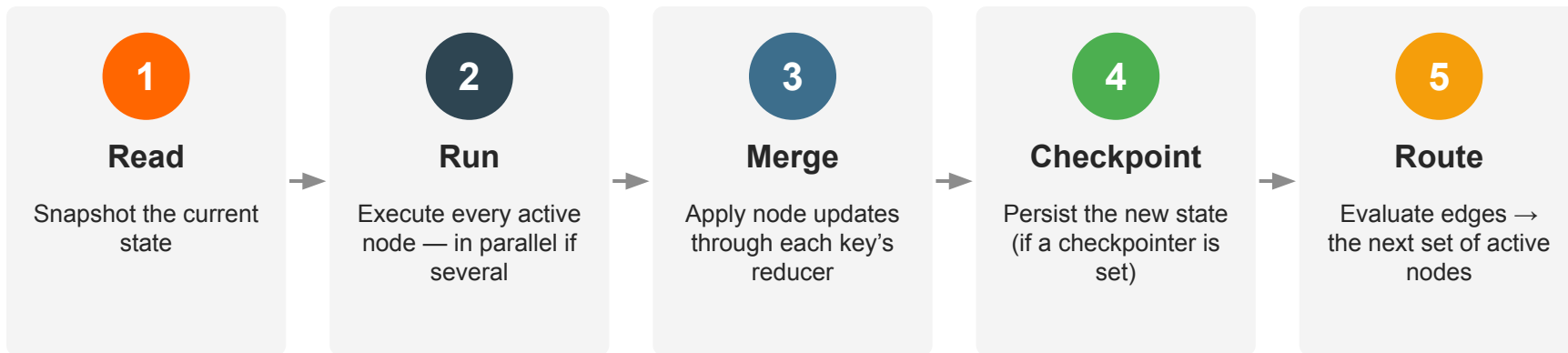
The classic tool-calling loop as a graph



conditional edge: tool calls?

yes → tools · no → END · tools always loops back

What happens when you call `invoke()`: super-steps



🔄 repeat until *END* is reached, an interrupt pauses the run, or the recursion limit is hit

Why super-steps?

The model is inspired by Google's Pregel (bulk-synchronous parallel). Nodes in the same step can't see each other's writes until the step ends — so parallel branches stay predictable.

Why reducers matter

With no reducer, an update overwrites the key. If two parallel nodes write the same key without a reducer, the step fails. Reducers (append, add, custom merge) make fan-in safe.

A tool-calling agent in ~20 lines

`agent_graph.py`

```
class State(TypedDict):
    messages: Annotated[list, add_messages]

def agent(state: State):
    reply = llm_with_tools.invoke(state["messages"])
    return {"messages": [reply]}

g = StateGraph(State)
g.add_node("agent", agent)
g.add_node("tools", ToolNode(tools))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)
g.add_edge("tools", "agent")

app = g.compile(checkpointer=InMemorySaver())
app.invoke(
    {"messages": [("user", "Refund order 42")]},
    {"configurable": {"thread_id": "t-1"}})
```

1

State + reducer

`add_messages` appends each node's messages to history

2

Node = function

Returns only the keys it changes

3

Routing

`tools_condition` sends to "tools" or END

4

Compile

Validates the graph and attaches persistence

5

Thread

`thread_id` scopes memory to one conversation

Persistence turns a graph into a durable agent



Checkpointers & threads

State is saved after every super-step, keyed by `thread_id`. Conversations resume, and a crashed run restarts from its last checkpoint — durable execution.



Human-in-the-loop

`interrupt()` inside a node pauses the graph and surfaces a value to a person. Resume later with `Command(resume=...)` — even days later, on another server.



Time travel

Every checkpoint is kept. Inspect the state history, replay from any point, or fork with an edited state to debug “what if the agent had...”.



Streaming

Stream full state values, per-node updates, LLM tokens, or custom events — so the UI can show progress as the graph runs.

Multi-agent systems are just bigger graphs



Supervisor

A router node (often an LLM) picks which worker runs next; every worker reports back to it.



Handoffs

A node returns `Command(goto="billing", update={...})` — it moves control and updates state in one step.



Dynamic fan-out

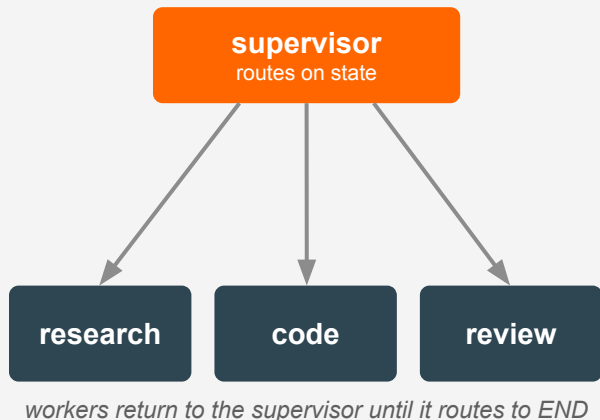
Return a list of `Send("worker", item)` to spawn one branch per item at runtime; a reducer merges results.



Subgraphs

A compiled graph can be a node in another graph — each agent team stays encapsulated and testable.

Supervisor pattern



Key point: LangGraph has no special “agent team” object. Supervisors, swarms and hierarchies are all built from the same state, nodes and edges — so you can see and test every hop.

02

CREWAI

Agents as a role-playing team



Four objects: agent, task, crew, process



Agent

Who does the work

- role
- goal
- backstory
- tools · llm
- allow_delegation



Task

What must be done

- description
- expected_output
- agent
- context (prior tasks)
- output_pydantic



Crew

The team that runs it

- agents
- tasks
- process
- memory
- kickoff(inputs)



Process

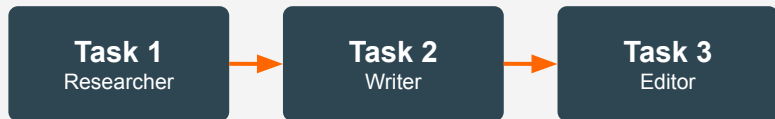
How work is ordered

- sequential — tasks in order
- hierarchical — a manager plans & delegates

The prompt is assembled for you: role + goal + backstory become the agent's persona; description + expected_output become its instructions.

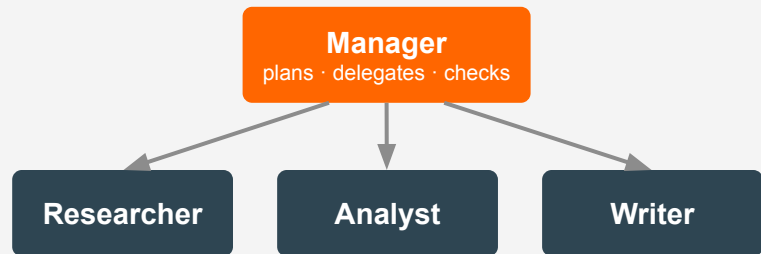
What happens when you call kickoff()

Sequential process



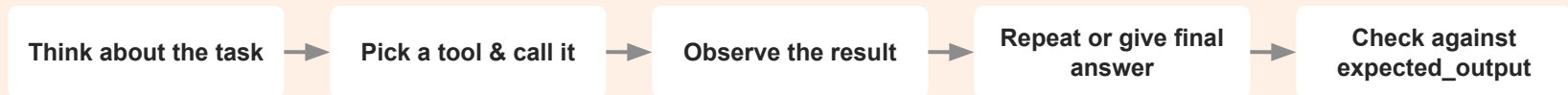
Each task's output is passed as context to the next. Order is the list order you gave the crew.

Hierarchical process



A manager (manager_llm or a custom manager_agent) assigns tasks and validates results before moving on.

Inside every task, the agent runs its own reasoning loop



A two-agent research crew

research_crew.py

```
researcher = Agent(
    role="Market Researcher",
    goal="Find 3 credible trends on {topic}",
    backstory="Ex-analyst who cites every claim",
    tools=[search_tool])
writer = Agent(
    role="Tech Writer", goal="Turn research into a crisp brief",
    backstory="Writes for busy CTOs")

research = Task(description="Research {topic}",
    expected_output="3 trends, each with a source",
    agent=researcher)
brief = Task(description="Write a one-page brief",
    expected_output="Markdown, under 300 words",
    agent=writer, context=[research])

crew = Crew(agents=[researcher, writer],
    tasks=[research, brief],
    process=Process.sequential, memory=True)
crew.kickoff(inputs={"topic": "agentic AI"})
```

1

Persona

role, goal and backstory steer behaviour

2

Templated inputs

{topic} is filled from kickoff(inputs)

3

Definition of done

expected_output tells the agent when to stop

4

Explicit hand-off

context=[research] feeds one task into another

5

Process & memory

One switch changes ordering and recall

Flows: taking back control around crews

`ticket_flow.py`

```
class Triage(Flow[TicketState]):
    @start()
    def classify(self):
        self.state.severity = score(self.state.ticket)

    @router(classify)
    def route(self):
        sev = self.state.severity
        return "urgent" if sev > 7 else "normal"

    @listen("urgent")
    def escalate(self):
        return SupportCrew().crew().kickoff(...)

    @listen("normal")
    def auto_reply(self): ...
```

@start()

Entry point(s) of the flow

@listen(x)

Runs when method x finishes or label x is emitted

@router(x)

Returns a label that picks the next branch

Typed state

A Pydantic model shared by every step

Crews = autonomy, Flows = control. Use plain Python and single LLM calls where you can, and drop into a crew only for the steps that need collaboration.

03

DECISION

Which one — and when to combine



How they compare

	LangGraph	CrewAI
Mental model	State machine / graph	Team of role-playing agents
You define	State schema, nodes, edges	Agents, tasks, process
Control flow	Explicit edges & routing functions	Process (or Flow decorators)
State	Typed state with reducers	Task outputs as context; Flow state
Multi-agent	Supervisor, handoffs, Send, subgraphs	Sequential or hierarchical crews
Human-in-the-loop	interrupt() + resume at any node	human_input on tasks; Flow steps
Persistence	Checkpointers, threads, time travel	Built-in memory; Flow state persistence
Learning curve	Steeper — more code, more control	Gentler — fast first prototype

Neither is “better” — they put control in different places.

Which framework should you reach for?



Choose LangGraph if...

- You need exact control over every branch and loop
- Runs are long, must survive crashes, or pause for approval
- You must debug, replay or audit each step
- The workflow is complex, stateful and production-bound



Choose CrewAI if...

- The work maps naturally onto roles and hand-offs
- You want a working prototype in an afternoon
- Business users should read the agent definitions
- Collaboration matters more than precise routing

Combine them: a crew can be called from inside a LangGraph node, and a LangGraph graph can sit inside a CrewAI Flow step. Let the outer layer own control and recovery; let the inner layer own collaboration.

Thank You

Questions & discussion

Trainer: Lakshminarayanan